

Gigalogy SQA Technical Assessment

Demo Shop Web Application API, UI Functional & Performance — Test Strategy

SQA technical assessment covering API testing, UI automation, performance testing, test cases, and bug reporting.

1. Project Overview

This repository contains the QA testing artifacts for the Demo Shop application, including API testing, Web UI functional testing, automation, bug reporting, and basic performance testing. The testing focuses on functional coverage, negative/edge-case testing, automation maintainability, performance testing, and clear defect reporting.

Application Under Test

Component	Details
API Endpoint	POST /v1/items/search
API URL	https://demo-shop-api.gigalogy.com.bd/v1/sandbox
Web UI	https://demo-shop.gigalogy.com.bd/

2. Tools & Technologies

Area	Tool
API Testing	Postman
UI Automation	Selenium
Programming Language	Python
Performance Testing	Postman Runner
Bug Tracking	Excel / Markdown
Version Control	Git & GitHub
Browser	Google Chrome

3. Test Strategy

Objective

The objective is to evaluate the Demo Shop application through API testing, Web UI functional testing, automation, defect reporting, and basic performance testing.

API Testing Approach

Black-box testing techniques were applied, including:

- Equivalence Partitioning

- Boundary Value Analysis
- Positive and negative testing
- Filter and sorting combination testing
- Response schema and data validation

API Test Coverage

The following scenarios were covered:

1. Valid keyword search
2. Empty/blank keyword
3. Price range filtering
4. Price sorting — ascending and descending
5. Review average sorting — ascending and descending
6. Review count sorting — ascending and descending
7. Invalid/nonexistent keyword
8. Invalid query parameters or filter combinations

Each API response is validated for status code, response structure, returned data, filtering/sorting behavior, and response time where applicable.

Web UI Testing Approach

The following user flows were tested:

- Product Search
- Product Details
- Add to Cart
- Cart Verification
- Pagination / Loading Additional Products

UI Automation

Two core flows were selected for automation, alongside a homepage sanity check:

- Homepage Load
- Product Search
- Add to Cart

Selenium WebDriver was selected because it provides reliable browser automation and supports interaction with web elements across modern browsers. Stable locators such as ID, Name, CSS Selector, XPath, and visible text are preferred where available to keep the automation scripts maintainable and reliable.

Bug Reporting

Each identified defect contains:

- Bug Title
- Preconditions

- Steps to Reproduce
- Expected Result
- Actual Result
- Severity
- Priority
- Evidence

Severity and Priority are assessed separately, based on technical impact and business urgency respectively.

Performance Testing

A basic performance check was performed against the API using approximately 50 requests. The following metrics are considered:

- Average response time
- p90 / p95 latency
- Throughput (RPS)
- Error percentage

This is a smoke-level performance check and should not be considered a full load or stress test.

4. Manual Test Cases

Manual test cases cover both API and Web UI functionality.

API

The API test cases cover:

- Keyword search
- Price range filtering
- Sorting
- Invalid inputs
- Response validation

Web UI

The Web UI test cases cover:

- Search
- Product details
- Add to cart
- Cart verification
- Pagination / additional product loading

The detailed test cases are available in the Test Cases and API Testing folders.

5. API Testing

API testing was performed using Postman. The Postman collection contains requests and automated assertions using Postman test scripts.

Example Validations

- HTTP status code
- Success response
- Response structure
- Product fields
- Search results
- Price range
- Sorting order
- Response time

Run API Tests

9. Open Postman.
10. Import the collection from: API Testing/Demo_Shop_API.postman_collection.json
11. Select the required environment/configuration if applicable.
12. Open "Collection Runner".
13. Select the collection.
14. Run the collection.
15. Review the test results and assertions.

6. UI Automation

UI automation is implemented using Selenium with Python.

Automated Flows

Homepage Load

16. Opens the Demo Shop website.
17. Verifies that all elements, images, and logos are displayed.

Search Flow

18. Opens the Demo Shop website.
19. Locates the search field.
20. Enters a valid search keyword.
21. Performs the search.
22. Verifies that search results are displayed.

Add to Cart Flow

23. Opens the Demo Shop website.
24. Searches for a product.

25. Selects a product.
26. Adds the product to the cart.
27. Verifies the cart count.
28. Verifies that the selected product is displayed in the cart.

7. Installation & Setup

Prerequisites

Requirement	Version
Python	3.12.6
Google Chrome	153.0.8010.53
Selenium	4.49.0

8. Bug Reports

Identified defects are documented with reproducible steps and supporting evidence. Each bug report includes:

- Bug ID
- Title
- Preconditions
- Steps to Reproduce
- Expected Result
- Actual Result
- Severity
- Priority

The detailed bug reports are available in: [Bug Reports/](#)

9. Performance Testing

A basic API performance test was conducted using 50 requests.

Metrics

Metric	Result
Total Requests	51
Average Response Time	1,114 ms
p90	1,631 ms
p95	2,454 ms
Throughput	0.84 requests/second

Metric	Result
Error Rate	0%

Detailed results are available in: Performance Testing/

10. Test Environment

- Public demo shop environment
- Google Chrome
- API tested through Postman
- UI automated using Selenium
- No authentication required
- Test data based on the available demo environment

11. Risks & Assumptions

Risks

- Demo data may change between test runs.
- The public environment may become temporarily unavailable.
- API responses may vary depending on the available dataset.
- Performance results may not represent production-level performance.

Assumptions

- POST /v1/items/search is the API endpoint under test.
- Expected behavior is based on the assignment requirements where no formal specification is provided.
- The performance test is intended as a basic performance check rather than a full capacity test.

12. Deliverables

#	Deliverable	Location
1	Test Strategy	README.md
2	API Test Cases	API Testing/
3	Web UI Test Cases	Test Cases/
4	API Testing	API Testing/
5	UI Automation	Automation/
6	UI Manual Testing	Manual/
7	Bug Reports	Bug Reports/
8	Performance Report	Performance Testing/

13. Conclusion

The testing activity covers the major functional areas requested in the assignment, including API validation, Web UI testing, automation, defect reporting, and basic performance testing. The repository is organized to keep test cases, automation code, bug reports, and performance results separated and easy to review.